```
# Cell 1: Import libraries and load Bhopal weather dataset

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import requests

from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier,
RandomForestRegressor
from sklearn.metrics import accuracy_score, classification_report
from sklearn.metrics import confusion_matrix, mean_absolute_error,
r2_score

# Load historical Bhopal weather data

url = "https://archive-api.open-meteo.com/v1/archive"

params = {
    "latitude": 23.26,
    "longitude": 77.41,
    "start_date": "2015-01-01",
    "end_date": "2024-12-31",
    "daily":
"temperature_2m_max,temperature_2m_min,precipitation_sum,wind_speed_10
m_max",
    "timezone": "Asia/Kolkata"
}

response = requests.get(url, params=params)
response.raise_for_status()

weather_data = response.json()["daily"]

df = pd.DataFrame(weather_data)

# Rename columns

df = df.rename(columns={
    "time": "date",
    "precipitation_sum": "precipitation",
    "temperature_2m_max": "temp_max",
    "temperature_2m_min": "temp_min",
    "wind_speed_10m_max": "wind"
})

df["date"] = pd.to_datetime(df["date"])

# Arrange columns
```

```
df = df[["date", "precipitation", "temp_max", "temp_min", "wind"]]

# Display first five records

print(df.head())

# Check missing values

print("\nMissing Values:")
print(df.isnull().sum())
```

```
        date  precipitation  temp_max  temp_min  wind
0 2015-01-01           15.4      17.0      13.4  17.4
1 2015-01-02            1.0      17.5      13.7  11.5
2 2015-01-03            0.0      20.2      14.2  10.0
3 2015-01-04            0.0      20.0      11.6  11.2
4 2015-01-05            0.0      20.4       8.4   8.7

Missing Values:
date             0
precipitation    0
temp_max         0
temp_min         0
wind             0
dtype: int64
```

```
# Cell 2: Data Analysis and Visualization

# Display dataset information
print("Dataset Information:")
print(df.info())

# Display statistical information
print("\nStatistical Description:")
print(df.describe())

# Create weather category
df['weather'] = np.where(
    df['precipitation'] > 0, 'Rain', 'No Rain'
)

# Count weather conditions
print("\nWeather Condition Counts:")
print(df['weather'].value_counts())

# Graph 1: Weather condition count
df['weather'].value_counts().plot(
    kind='bar',
    title='Bhopal Weather Condition Counts'
)
```

```
plt.xlabel('Weather Condition')
plt.ylabel('Number of Days')
plt.show()

# Graph 2: Temperature over time
df.set_index('date')[['temp_max', 'temp_min']].plot(
    figsize=(12, 4),
    title='Bhopal Temperature Over Time'
)

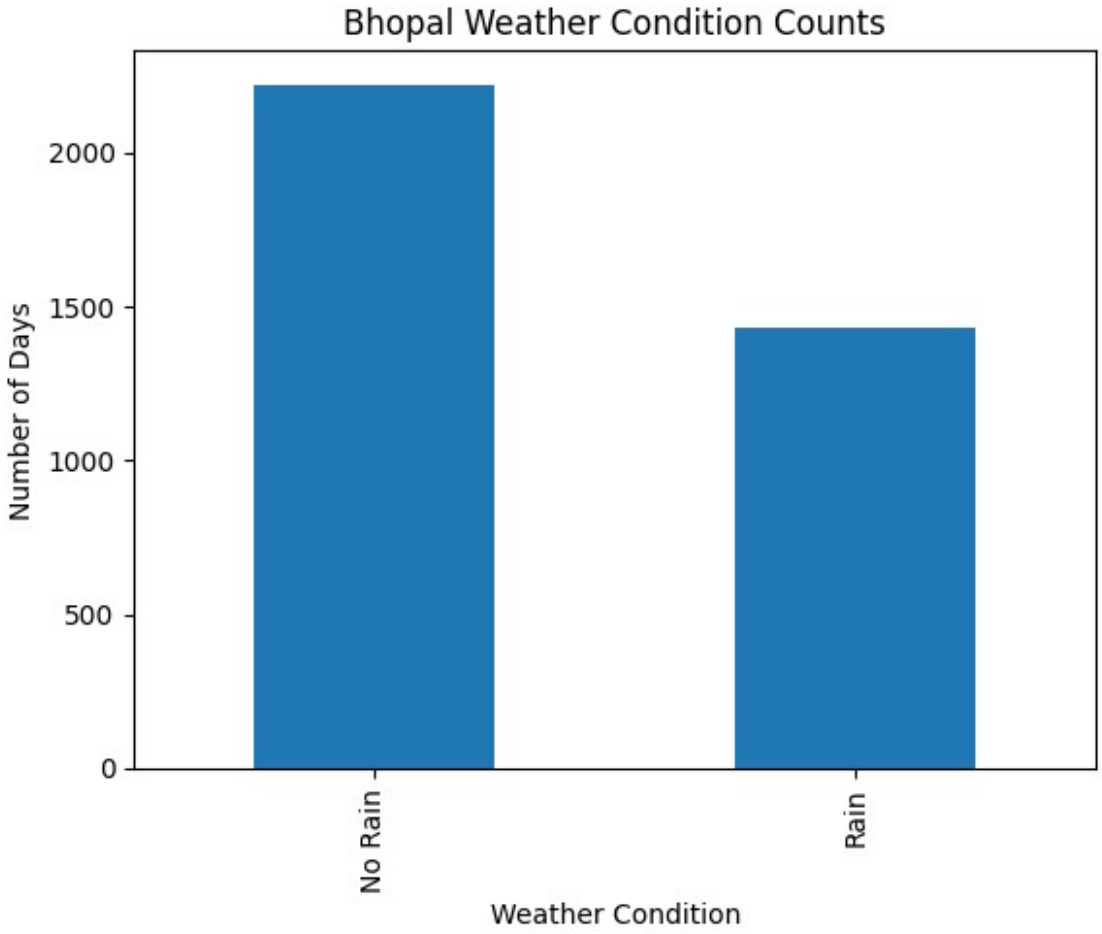
plt.xlabel('Date')
plt.ylabel('Temperature (°C)')
plt.show()
```

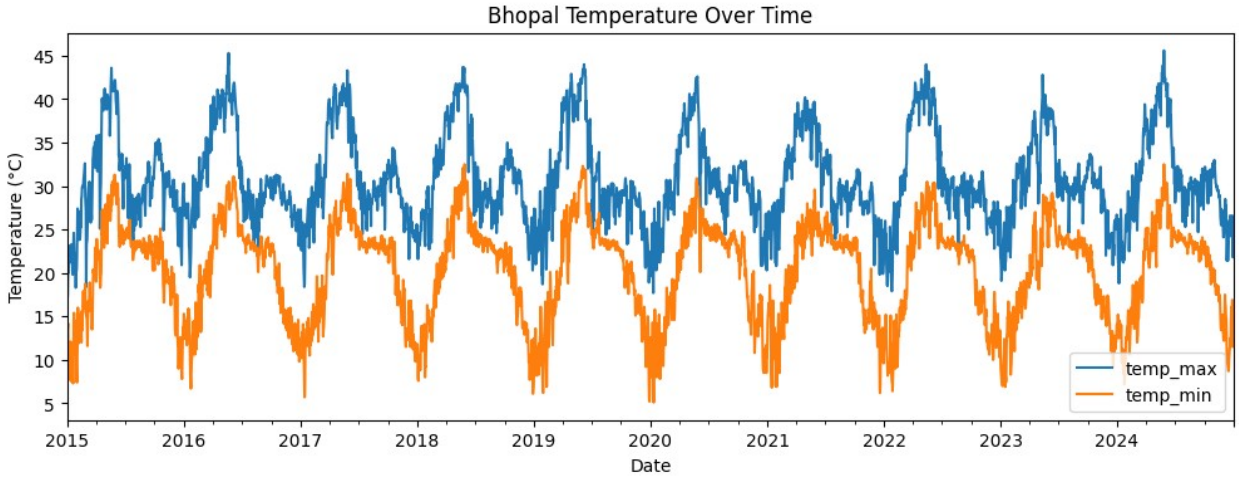
```
Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3653 entries, 0 to 3652
Data columns (total 5 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   date           3653 non-null   datetime64[ns]
 1   precipitation  3653 non-null   float64
 2   temp_max       3653 non-null   float64
 3   temp_min       3653 non-null   float64
 4   wind           3653 non-null   float64
dtypes: datetime64[ns](1), float64(4)
memory usage: 142.8 KB
None

Statistical Description:
                      date  precipitation     temp_max     temp_min  \
count                 3653    3653.000000  3653.000000  3653.000000
mean   2020-01-01 00:00:00       3.789707    30.783028    20.180701
min    2015-01-01 00:00:00       0.000000    17.000000     5.100000
25%    2017-07-02 00:00:00       0.000000    27.300000    15.500000
50%    2020-01-01 00:00:00       0.000000    29.900000    22.000000
75%    2022-07-02 00:00:00       1.600000    33.700000    24.000000
max    2024-12-31 00:00:00     283.600000    45.600000    32.500000
std                    NaN      12.008033     5.343311     5.655299

              wind
count  3653.000000
mean     14.902546
min       5.200000
25%      11.200000
50%      14.000000
75%      18.000000
max      42.000000
std       4.663138
```

```
Weather Condition Counts:
weather
No Rain    2221
Rain       1432
Name: count, dtype: int64
```






```python
# Cell 3: Train Random Forest Classification Model

# Create month feature
df['month'] = df['date'].dt.month

# Select input features
X = df[['temp_max', 'temp_min', 'wind', 'month']]

# Select target variable
y = df['weather']

# Split dataset into training and testing data
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

# Create Random Forest model
model = RandomForestClassifier(
    n_estimators=100,
    random_state=42
)

# Train the model
model.fit(X_train, y_train)

# Make predictions
y_pred = model.predict(X_test)

# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)

print("Model Training Completed Successfully!")
```

```
print("Training Data:", len(X_train))
print("Testing Data:", len(X_test))
print("Model Accuracy:", accuracy * 100, "%")

# Display classification report
print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```

```
Model Training Completed Successfully!
Training Data: 2922
Testing Data: 731
Model Accuracy: 90.01367989056087 %

Classification Report:
              precision    recall  f1-score   support

     No Rain       0.90      0.93      0.92       444
        Rain       0.89      0.85      0.87       287

    accuracy                           0.90       731
   macro avg       0.90      0.89      0.89       731
weighted avg       0.90      0.90      0.90       731
```

```
# Cell 4: Confusion Matrix

from sklearn.metrics import confusion_matrix
from sklearn.metrics import ConfusionMatrixDisplay

# Calculate confusion matrix
cm = confusion_matrix(
    y_test,
    y_pred,
    labels=['No Rain', 'Rain']
)

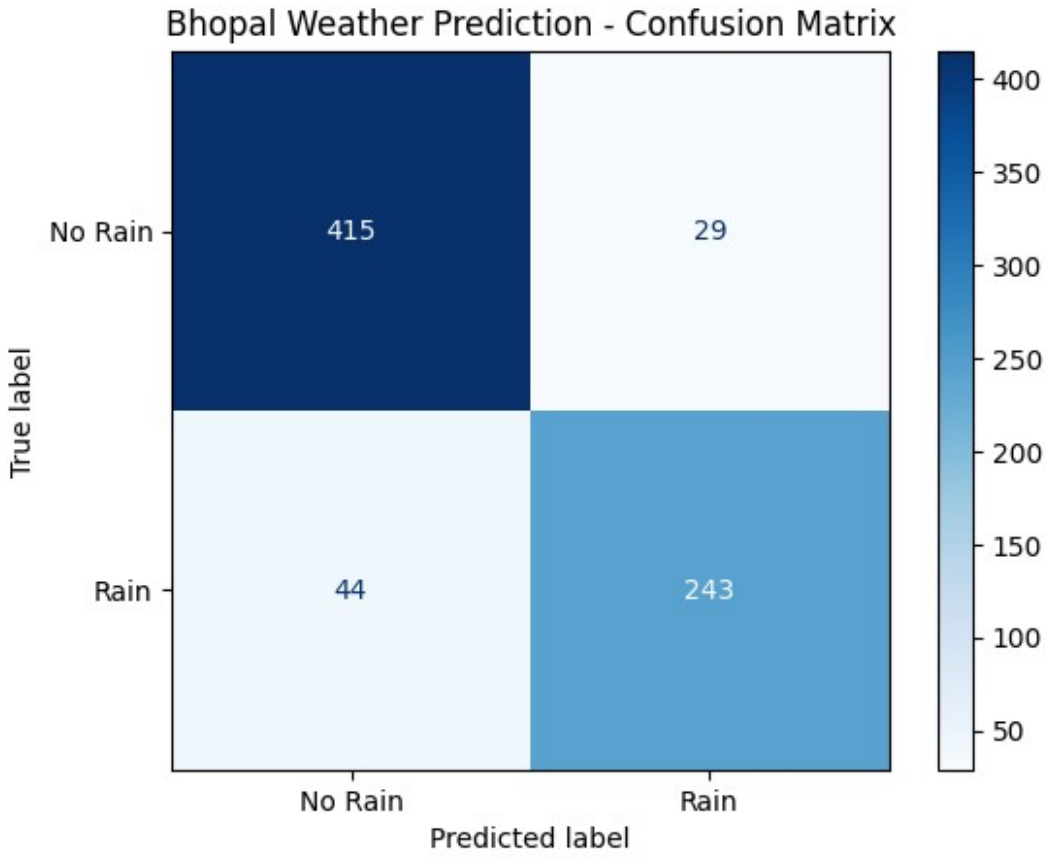
# Display confusion matrix
print("Confusion Matrix:")
print(cm)

# Visualize confusion matrix
disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=['No Rain', 'Rain']
)

disp.plot(cmap='Blues')

plt.title("Bhopal Weather Prediction - Confusion Matrix")
plt.show()
```

```
Confusion Matrix:
[[415  29]
 [ 44 243]]
```




```python
# Next Day Temperature Prediction

ts = df.sort_values('date').reset_index(drop=True).copy()

ts['month'] = ts['date'].dt.month

for lag in [1, 2, 3]:
    for col in ['temp_max', 'temp_min',
                'precipitation', 'wind']:
        ts[f'{col}_lag{lag}'] = ts[col].shift(lag - 1)

ts['target_temp_max_tomorrow'] = ts['temp_max'].shift(-1)

ts = ts.dropna().reset_index(drop=True)

feat_cols = [c for c in ts.columns if '_lag' in c] + ['month']

X = ts[feat_cols]
y = ts['target_temp_max_tomorrow']
```

```
split = int(len(ts) * 0.8)

X_train, X_test = X.iloc[:split], X.iloc[split:]
y_train, y_test = y.iloc[:split], y.iloc[split:]

from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, r2_score

reg = RandomForestRegressor(
    n_estimators=300,
    random_state=42
)

reg.fit(X_train, y_train)

pred = reg.predict(X_test)

print("MAE:", mean_absolute_error(y_test, pred))
print("R2 Score:", r2_score(y_test, pred))
```

```
MAE: 1.0191086757990877
R2 Score: 0.9241689194222275
```

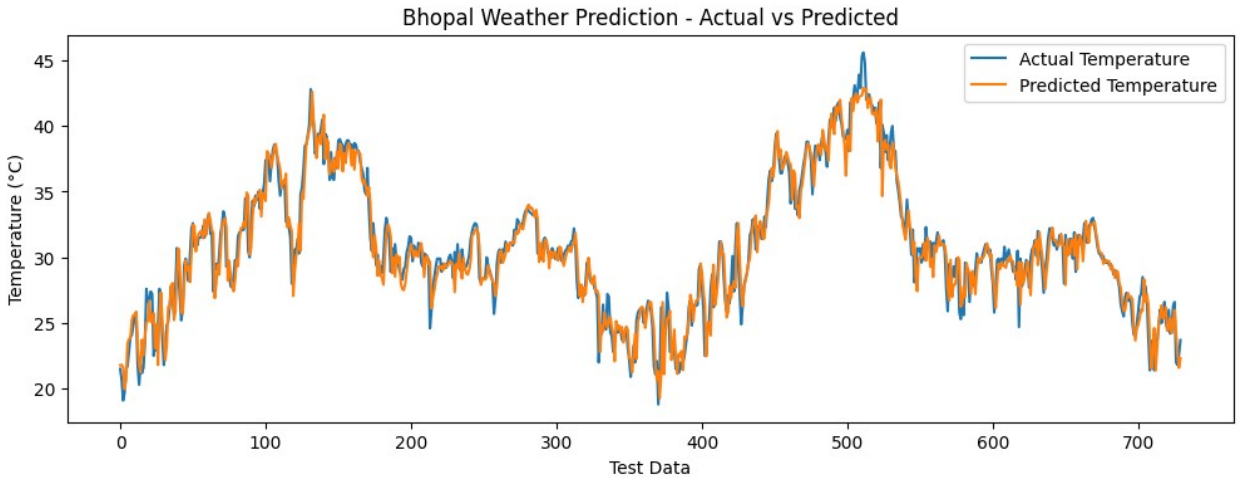
```
import matplotlib.pyplot as plt

plt.figure(figsize=(12, 4))

plt.plot(y_test.values, label='Actual Temperature')
plt.plot(pred, label='Predicted Temperature')

plt.title('Bhopal Weather Prediction - Actual vs Predicted')
plt.xlabel('Test Data')
plt.ylabel('Temperature (°C)')

plt.legend()
plt.show()
```




```
# Take user input
print("\n===== BHOPAL WEATHER PREDICTION =====")

temp_max = float(input("Enter Maximum Temperature (°C): "))
temp_min = float(input("Enter Minimum Temperature (°C): "))
precipitation = float(input("Enter Precipitation: "))
wind = float(input("Enter Wind Speed: "))
month = int(input("Enter Month (1-12): "))

# Create data using original training features
input_data = {
    'precipitation': precipitation,
    'temp_max': temp_max,
    'temp_min': temp_min,
    'wind': wind,
    'month': month
}

custom_data = pd.DataFrame([input_data])

# Use the exact columns expected by the model
custom_data = custom_data[model.feature_names_in_]

# Predict weather
rain_prediction = model.predict(custom_data)

print("\n===== WEATHER PREDICTION =====")

if rain_prediction[0] == 1:
    print("Rain is expected.")
else:
    print("No Rain is expected.")

print("\nCustom weather prediction completed!")
```

```
===== BHOPAL WEATHER PREDICTION =====
Enter Maximum Temperature (°C): 29
Enter Minimum Temperature (°C): 20
Enter Precipitation: 0
Enter Wind Speed: 10
Enter Month (1-12): 11

===== WEATHER PREDICTION =====
No Rain is expected.

Custom weather prediction completed!
```